实验 5 数组实验

5.1 实验目的

（1）掌握数组的说明、初始化和使用。

（2）掌握一维数组作为函数参数时实参和形参的用法。

（3）掌握字符串处理函数的设计，包括串操作函数及数字串与数之间转换函数实现算法。

（4）掌握基于分治策略的二分查找算法和选择法排序算法的思想，以及相关算法的实现。

5.2 实验内容及要求

1、 源程序改错与跟踪调试

在下面所给的源程序中，函数 strcate(t,s)的功能是将字符串 s 连接到字符串 t 的尾部；函数 strdelc(s,c)的功能是从字符串 s 中删除所有与给定字符 c 相同的字符，程序应该能够输出如下结果：

Programming Language

ProgrammingLanguage Language

ProgrmingLnguge

跟踪和分析源程序中存在的问题，排除程序中的各种逻辑错误，使之能够输出正确的结果。

（1） 单步执行源程序。进跟踪进入 strcate 时，观察字符数组 t 和 s 中的内容，分析结果是否正确。当单步执行光条刚落在第二个 while 语句所在行时，i 为何值？t[i]为何值？分析该结果是否存在问题。当单步执行光条落在 strcate 函数块结束标记即右花括号“}”所在行时，字符数组 t 和 s 分别为何值？分析是否实现了字符串连接。

解答：

1. i = 12
2. t[i] = \0
3. 存在问题，t = "Programming"，s = "Language"
4. 未实现字符串拼接

（2） 跟踪进入函数 strdelc 时，观察字符数组 s 中的内容和字符 c 的值，分析结果是否正确。单步执行 for 语句过程中，观察字符数组 s, j 和 k 值的变化，分析该结果是否存在问题。当单步执行光条落在 strdelc 函数块结束标记“}”所在行时，字符串 s 为何值？分析是否实现了所要求的删除操作。

解答：

1. s = "Programming", c = "a"
2. s 中的"a"会被替换掉，j 一直加 1，k 在 s[j] != "a"的时候加 1
3. s = "Progrmmingg"
4. 未实现

```
/*实验 5 程序改错与跟踪调试题程序*/
#include<stdio.h>
void strcate(char [],char []);
void strdelc(char [],char );
int main(void)
{
        char a[]="Language", b[]="Programming";
        printf("%s %s\n", b,a);
        strcate(b,a);  printf("%s %s\n",b,a);
        strdelc(b, 'a');      printf("%s\n",b);
        return 0;
}
void strcate(char t[],char s[])
{
        int i = 0,    j = 0;
        while(t[i++]) ;
        while((t[i++] = s[j++] )!= '\0');
}
void strdelc(char s[], char c)
{
        int j,k;
        for(j=k=0; s[j] != '\0'; j++)
            if(s[j] != c)   s[k++] = s[j];
}
```

解答：

修改后的源代码：

```
#include <stdio.h>
void strcate(char[], char[]);
void strdelc(char[], char);
int main(void)
{
    char a[] = "Language", b[] = "Programming";
    printf("%s %s\n", b, a);
    strcate(b, a);
    printf("%s %s\n", b, a);
    strdelc(b, 'a');
    printf("%s\n", b);
    return 0;
}
void strcate(char t[], char s[])
{
    int i = 0, j = 0;
    while (t[i] != '\0')
        i++;
    while ((t[i++] = s[j++]) != '\0')
        ;
}
void strdelc(char s[], char c)
{
    int j, k;
    for (j = k = 0; s[j] != '\0'; j++)
        if (s[j] != c)
            s[k++] = s[j];
    s[k] = '\0';
}
```

2、 源程序完善和修改替换

(1) 下面的源程序用于求解约瑟夫问题：M 个人围成一圈，从第一个人开始依次从 1 至 N 循环报数，每当报数为 N 时报数人出圈，直到圈中只剩下一个

人为止。①请在源程序中的下划线处填写合适的代码来完善该程序。

```
#include<stdio.h>
#define M 10
#define N 3
int main(void)
{
    int a[M], b[M];   /* 数组 a 存放圈中人的编号，数组 b 存放出圈人的编号 */
    int i, j, k;
    for(i = 0; i < M; i++)            /* 对圈中人按顺序编号 1—M */
        a[i] = i + 1;
   for(i = M, j = 0; i > 1; i--){
/* i 表示圈中人个数，初始为 M 个，剩 1 个人时结束循环；j 表示当前报数人的位置 */
        for(k = 1; k <= N; k++)              /* 1 至 N 报数 */
            if(++j > i - 1) j = 0;/* 最后一个人报数后第一个人接着报，形成一个圈 */
            b[M-i] = j ? _______:______; /* 将报数为 N 的人的编号存入数组 b */
            if(j)
                for(k = --j; k < i; k++)     /* 压缩数组 a，使报数为 N 的人出圈 */
                        _______________;
    }
    for(i = 0;i < M-1; i++)          /* 按次序输出出圈人的编号 */
        printf("%6d", b[i]);
    printf("%6d\n", a[0]);            /* 输出圈中最后一个人的编号 */
    return 0;
}
```

解答：

源代码：

```
#include <stdio.h>
#define M 10
#define N 3
int main(void)
{
    int a[M], b[M]; /* 数组 a 存放圈中人的编号，数组 b 存放出圈人的编号 */
```

```
    int i, j, k;
    for (i = 0; i < M; i++) /* 对圈中人按顺序编号 1—M */
        a[i] = i + 1;
    for (i = M, j = 0; i > 1; i--)
    {
        /* i 表示圈中人个数，初始为M 个，剩 1 个人时结束循环；j 表示当前报数人的位置 */
        for (k = 1; k <= N; k++) /* 1 至 N 报数 */
            if (++j > i - 1)
                j = 0;                          /* 最后一个人报数后第一个人接着报，形成一个圈 */
        b[M - i] = j ? a[j - 1] : a[i - 1]; /* 将报数为N 的人的编号存入数组 b */
        if (j)
            for (k = --j; k < i; k++) /* 压缩数组 a，使报数为N 的人出圈 */
                a[k] = a[k + 1];
    }
    for (i = 0; i < M - 1; i++) /* 按次序输出出圈人的编号 */
        printf("%6d", b[i]);
    printf("%6d\n", a[0]); /* 输出圈中最后一个人的编号 */
    return 0;
}
```

②上面的程序中使用数组元素的值表示圈中人的编号，故每当有人出圈时都要压缩数组，这种算法不够精炼。如果采用做标记的办法，即每当有人出圈时对相应数组元素做标记，从而可省掉压缩数组的时间，这样处理效率会更高一些。请采用做标记的办法修改程序，并使修改后的程序与原程序具有相同的功能。

解答：

源代码：

```
#include <stdio.h>
#define M 10
#define N 3
int main(void)
{
    int a[M], b[M];
    int i, j, k;
```

```
    for (i = 0; i < M; i++) /* 全部标记为未出圈 */
        a[i] = 1;
    for (i = M, j = 0; i > 1; i--)
    {
        /* i 表示圈中人个数，初始为M 个，剩1 个人时结束循环；j 表示当前报数人的位置 */
        for (k = 1; k <= N; k++) /* 1 至N 报数 */
        {
            if (++j > M)
            {
                j = 1; /* 最后一个人报数后第一个人接着报，形成一个圈 */
            }
            if (a[j - 1] == 0)
            {
                k--;
            }
        }
        b[M - i] = j; /* 将报数为N 的人的编号存入数组b */
        a[j - 1] = 0; /* 标记报数为N 的人已出圈 */
    }
    for (i = 0; i < M - 1; i++) /* 按次序输出出圈人的编号 */
        printf("%6d", b[i]);
    for (i = 0; i < M; i++) /* 找到最后剩下的人 */
        if (a[i] == 1)
            break;
    printf("%6d\n", i + 1); /* 输出圈中最后一个人的编号 */
    return 0;
}
```

3. 程序设计

以下（1）至（3）题对应 Educoder 教学平台 “C 语言实验”课程，实验 5，第 10 关实验 5-1、第 11 关实验 5-2，以及第 12 关实验 5-3。

（1）输入一个整数，将它在内存中二进制表示的每一位转化成对应的数字字符并且存放到一个字符数组中，然后输出该整数的二进制表示。

1）解答思路：算法流程如图 5-1 所示：

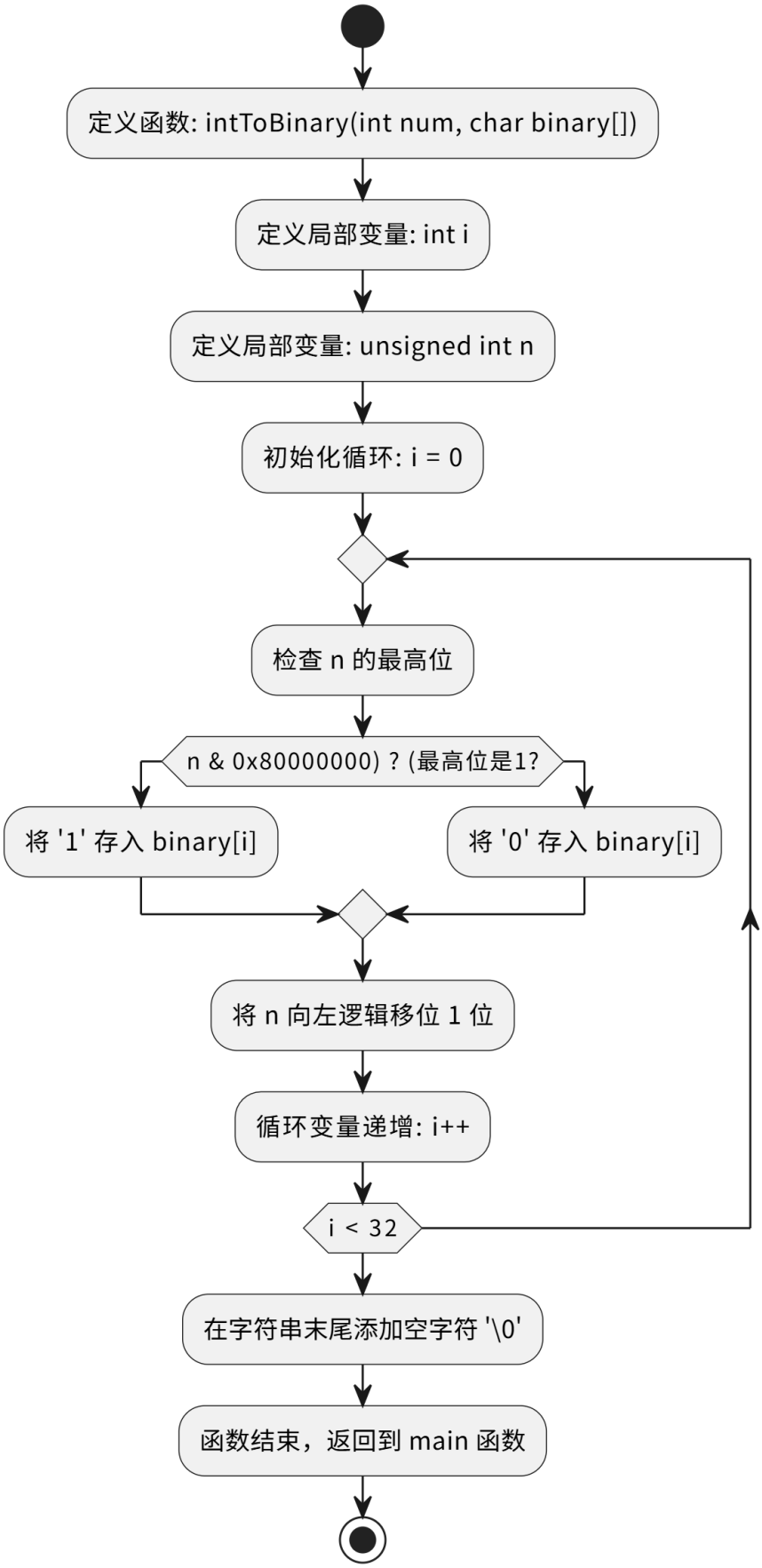

图 4-1 编程题 1 的程序流程图

2）源代码清单：

```
#include <stdio.h>
#include <string.h>

void intToBinary(int num, char binary[]);

int main(void)
{
    int num;
    char binary[33];

    printf("请输入一个整数: ");
    scanf("%d", &num);

    intToBinary(num, binary);

    printf("该整数的二进制表示为: %s\n", binary);

    return 0;
}

void intToBinary(int num, char binary[])
{
    int i;
    unsigned int n = (unsigned int)num;

    for (i = 0; i < 32; i++)
    {
        if (n & 0b10000000000000000000000000000000)
            binary[i] = '1';
        else
            binary[i] = '0';

        n <<= 1;
    }

    binary[32] = '\0';
}
```

3）测试

（a）测试数据：

如表 5-1 所示。

表 5-1 编程题 1 的测试数据

测试用例	程序输入	理论结果	实际结果
用例 1	127	00000000000000000000000000 1111111	00000000000000000000000000 1111111
用例 2	-1	11111111111111111111111111 11111	11111111111111111111111111 11111
用例 3	0	00000000000000000000000000 0000000	00000000000000000000000000 0000000

（b）对应测试测试用例的运行结果如图 5-2 所示。

图 5-2 编程题 1 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（2）编写一个 C 程序，要求采用模块化程序设计思想，将相关功能用函数实现，并提供菜单选项。该程序具有以下功能：

①“成绩输入”，输入 n 个学生的姓名和 C 语言课程的成绩。

②“成绩排序”，将成绩按从高到低的次序排序，姓名同时进行相应调整。

③“成绩输出”，输出排序后所有学生的姓名和 C 语言课程的成绩。

④“成绩查找”，输入一个 C 语言课程成绩值，用二分查找进行搜索。如果查找到有该成绩，则输出该成绩学生的姓名和 C 语言课程的成绩；否则，输出提示“not found!”。

解答：

1） 解题思路：算法流程如图 5-3，5-4，5-5，5-6，5-7，5-8 所示。

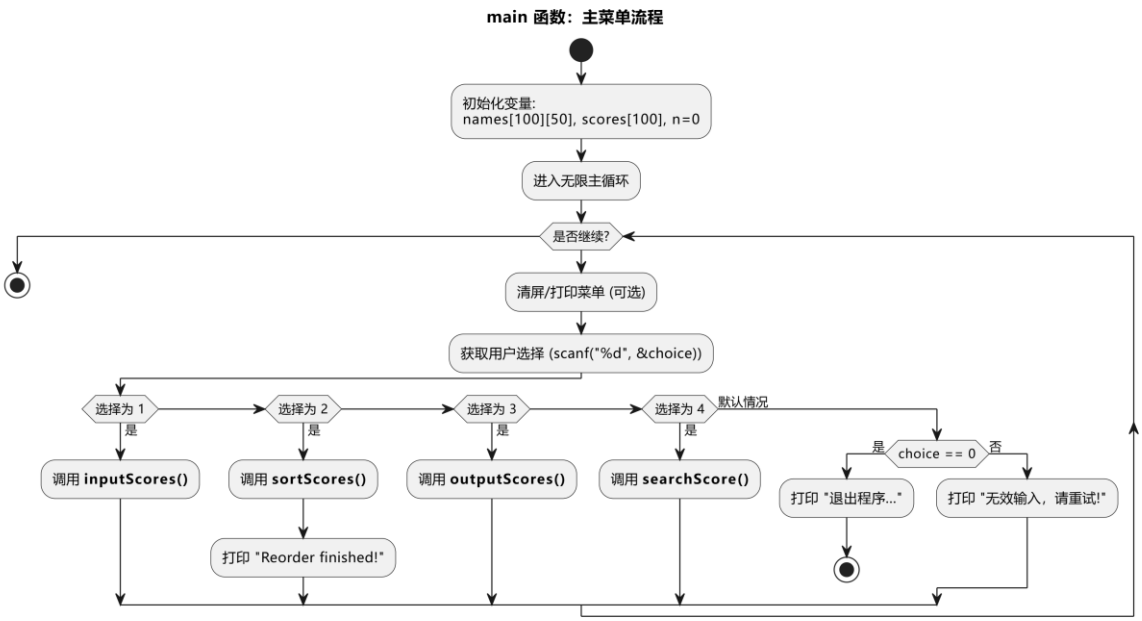

图 5-3 编程题 2 的程序流程图 1

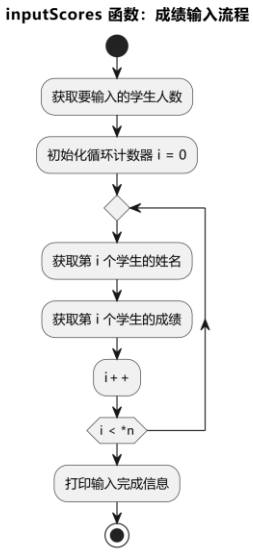

图 5-4 编程题 2 的程序流程图 2

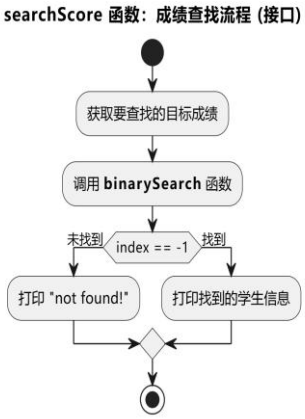

图 5-5 编程题 2 的程序流程图 3

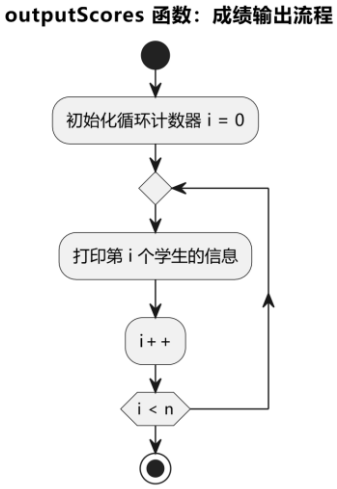

图 5-6 编程题 2 的程序流程图 4

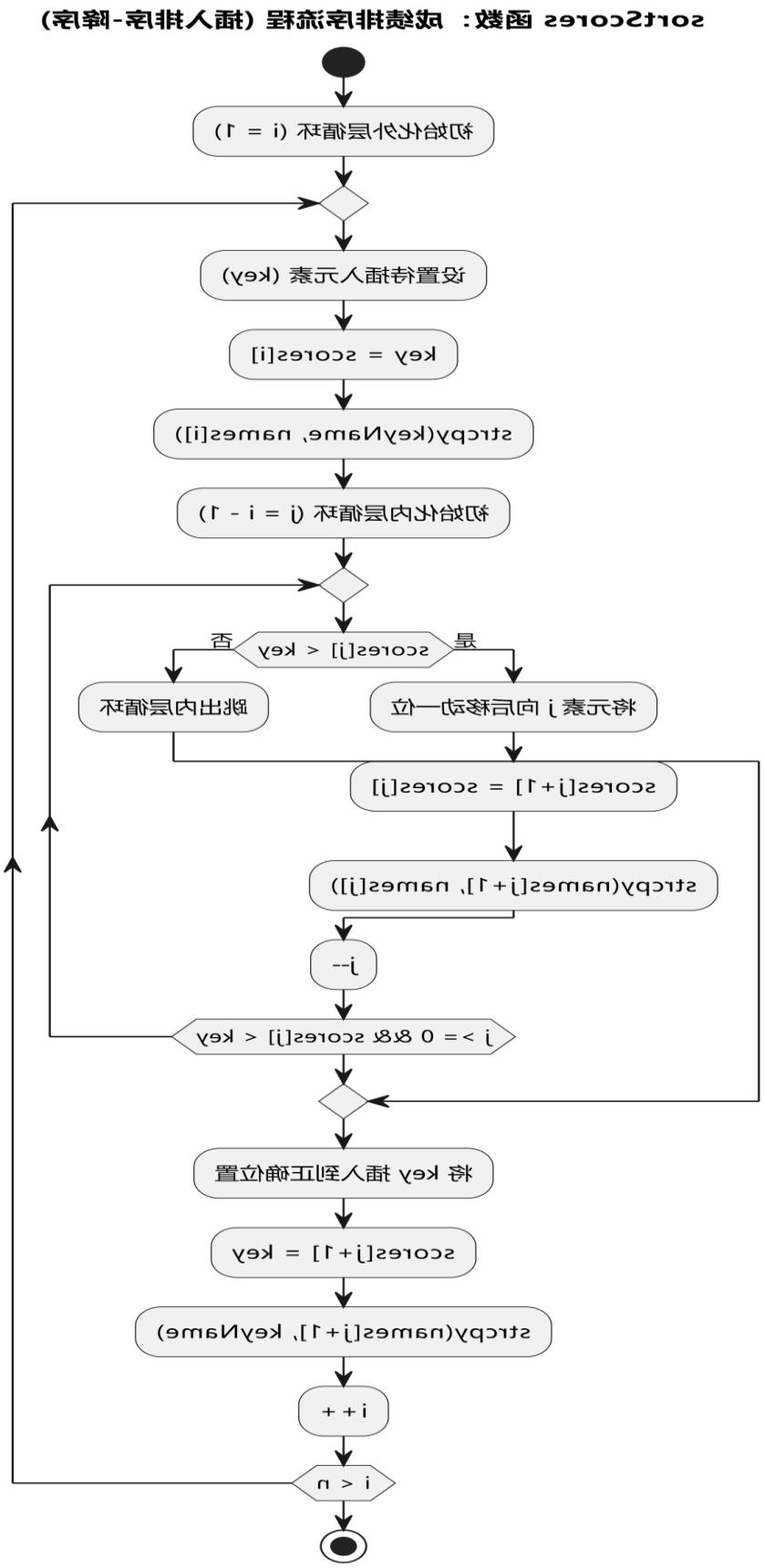

图 5-7 编程题 2 的程序流程图 5

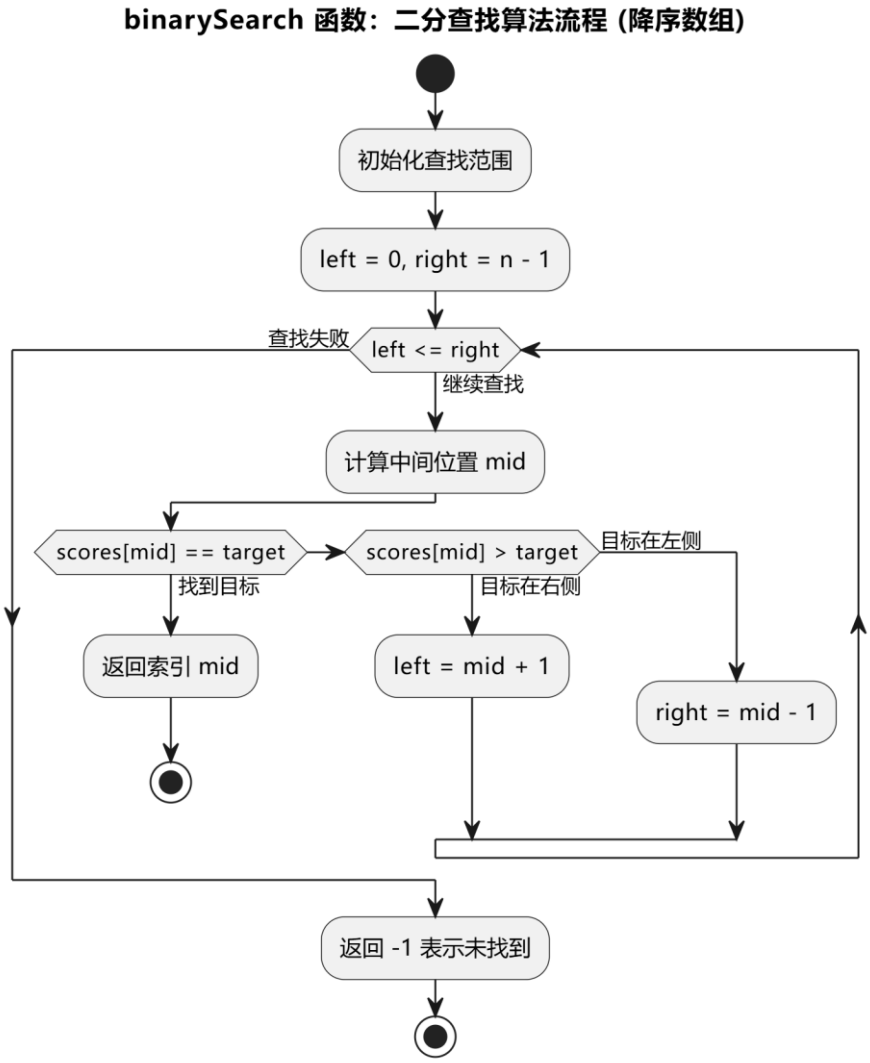

图 5-8 编程题 2 的程序流程图 6

2） 源代码清单：

```
#include <stdio.h>
#include <string.h>

void inputScores(char names[][50], int scores[], int *n);
void sortScores(char names[][50], int scores[], int n);
void outputScores(char names[][50], int scores[], int n);
void searchScore(char names[][50], int scores[], int n);
int binarySearch(int scores[], int n, int target);

int main(void)
{
    char names[100][50]; /* 学生姓名数组 */
```

```
    int scores[100];        /* 学生成绩数组 */
    int n = 0;              /* 学生人数 */
    int choice;

    while (1)
    {
        scanf("%d", &choice);

        switch (choice)
        {
        case 1:
            inputScores(names, scores, &n);
            break;
        case 2:
            sortScores(names, scores, n);
            printf("Reorder finished!\n");
            break;
        case 3:
            outputScores(names, scores, n);
            break;
        case 4:
            searchScore(names, scores, n);
            break;
        case 0:
            return 0;
        }
    }

    return 0;
}

/* 成绩输入函数 */
void inputScores(char names[][50], int scores[], int *n)
{
    int i;
    scanf("%d", n);

    for (i = 0; i < *n; i++)
    {
        scanf("%s %d", names[i], &scores[i]);
```

```
    }

    printf("%d records were input!\n", *n);
}

/* 成绩排序函数（从高到低） */
void sortScores(char names[][50], int scores[], int n)
{
    for (int i = 1; i < n; i++)
    {
        int key = scores[i];
        char keyName[50];
        strcpy(keyName, names[i]);
        int j = i - 1;
        for (; j >= 0 && scores[j] < key; j--)
        {
            scores[j + 1] = scores[j];
            strcpy(names[j + 1], names[j]);
        }
        scores[j + 1] = key;
        strcpy(names[j + 1], keyName);
    }
}

/* 成绩输出函数 */
void outputScores(char names[][50], int scores[], int n)
{
    int i;

    for (i = 0; i < n; i++)
    {
        printf("%s %d\n", names[i], scores[i]);
    }
}

/* 成绩查找函数 */
void searchScore(char names[][50], int scores[], int n)
{
    int target, index;
```

```
    scanf("%d", &target);

    index = binarySearch(scores, n, target);

    if (index == -1)
    {
        printf("not found!\n");
    }
    else
    {

        printf("%s %d\n", names[index], scores[index]);
    }
}

int binarySearch(int scores[], int n, int target)
{
    int left = 0, right = n - 1, mid;

    while (left <= right)
    {
        mid = (left + right) / 2;

        if (scores[mid] == target)
            return mid;
        else if (scores[mid] > target)
            left = mid + 1;
        else
            right = mid - 1;
    }

    return -1;
}
```

3） 测试

（a） 测试数据：

如表 5-2 所示：

测试用例	程序输入	理论结果	实际结果
用例 1	1	6 records were	6 records were

	6 Jack 95 Mike 90 Joe 75 Andy 95 Rose 89 Sophia 77	input!	input!
用例 2	2	Reorder finished!	Reorder finished!
用例 3	3	Jack 95 Andy 95 Mike 90 Rose 89 Sophia 77 Joe 75	Jack 95 Andy 95 Mike 90 Rose 89 Sophia 77 Joe 75
用例 4	4 89	Rose 89	Rose 89
用例 5	4 88	not found!	not found!
用例 6	0	（程序退出）	（程序退出）

（b）对应测试测试用例的运行结果如图 5-9 所示。

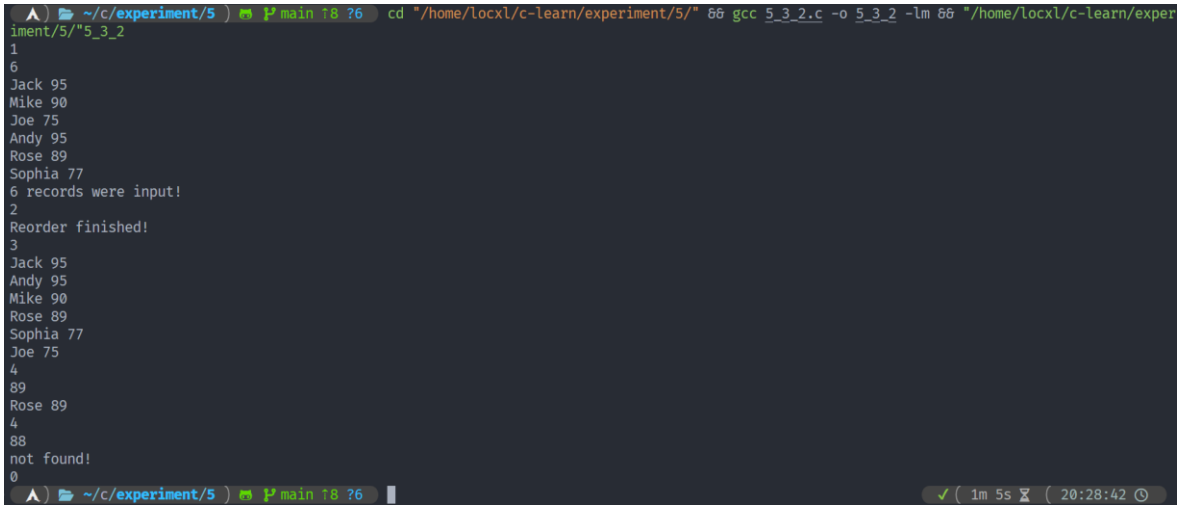

图 5-9 编程题 2 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

（3）求解 N 皇后问题，即在 N×N 的棋盘上摆放 N 个皇后，要求任意两个皇

后不能在同一行、同一列、同一对角线上。输入棋盘的大小 N（N 取值 1-10），如果能满足摆放要求，则输出所有可能的摆放法的数量，否则输出“无解。”

解答：

1）解答思路：算法流程如图 5-10 所示：

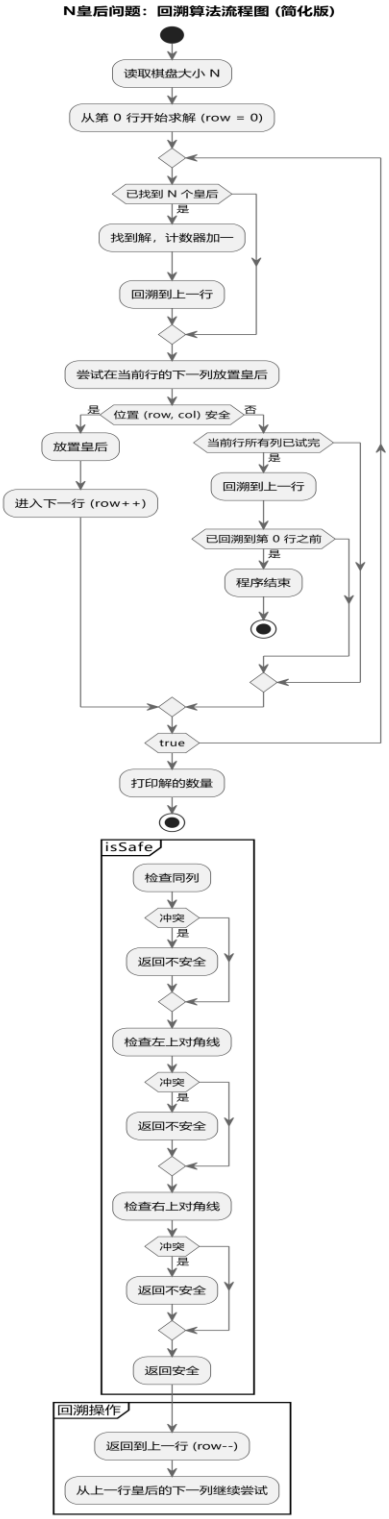

图 5-10 编程题 3 的程序流程图

2）源代码清单：

```
#include <stdio.h>
#include <stdlib.h>

#define MAX_N 10

int board[MAX_N];          /* board[i]表示第 i 行皇后所在的列号 */
int solutionCount = 0; /* 解的数量 */

/* 函数声明 */
int isSafe(int row, int col, int n);
void solveNQueens(int row, int n);
void printBoard(int n);

int main(void)
{
    int n;

    scanf("%d", &n);

    solutionCount = 0;
    solveNQueens(0, n);

    if (solutionCount == 0)
    {
        printf("无解\n");
    }
    else
    {
        printf("%d\n", solutionCount);
    }

    return 0;
}
```

```
/* 检查在(row, col)位置放置皇后是否安全 */
int isSafe(int row, int col, int n)
{
    int i;

    /* 检查同一列是否有皇后 */
    for (i = 0; i < row; i++)
    {
        if (board[i] == col)
            return 0;
    }

    /* 检查左上对角线是否有皇后 */
    for (i = row - 1; i >= 0; i--)
    {
        if (board[i] == col - (row - i))
            return 0;
    }

    /* 检查右上对角线是否有皇后 */
    for (i = row - 1; i >= 0; i--)
    {
        if (board[i] == col + (row - i))
            return 0;
    }

    return 1;
}

/* 使用回溯法求解N皇后问题 */
void solveNQueens(int row, int n)
{
    int col;
```

```
        /* 如果所有行都已放置皇后，找到一个解 */
        if (row == n)
        {
            solutionCount++;
            return;
        }

        /* 尝试在当前行的每一列放置皇后 */
        for (col = 0; col < n; col++)
        {
            if (isSafe(row, col, n))
            {
                board[row] = col;              /* 放置皇后 */
                solveNQueens(row + 1, n); /* 递归处理下一行 */
            }
        }
    }
```

3）测试

（a）测试数据：

如表 5-3 所示。

测试用例	程序输入	理论结果	实际结果
用例 1	1	1	1
用例 2	2	无解	无解
用例 3	8	92	92
用例 4	12	10	10

（b）对应测试测试用例的运行结果如图 5-11 所示。

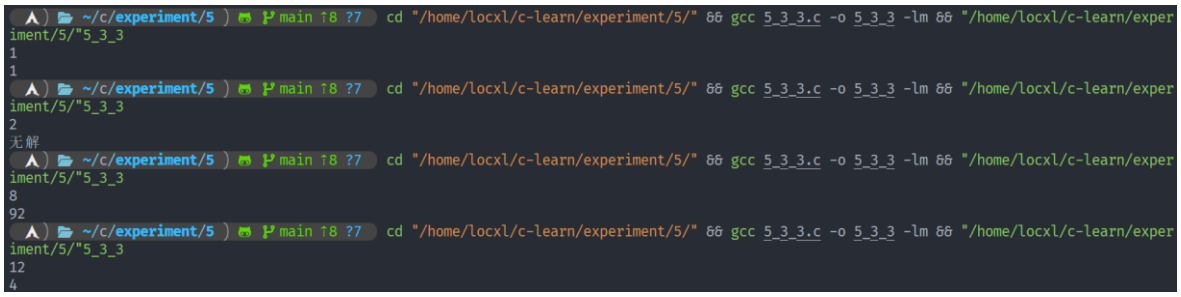

图 5-11 编程题 1 测试用例的运行结果

说明上述的运行结果与理论分析吻合，验证了程序的正确性。

5.3 实验小结

在本次实验中，我遇到了多个问题，如字符串连接函数 strcate 未能正确实现，导致字符串拼接失败；以及 strdelc 函数在删除字符时未正确处理数组边界，造成数据残留。通过逐行调试和分析程序逻辑，发现 strcate 中循环条件存在错误，应改为判断字符是否为结束符而非递增后判断，同时在 strdelc 中需在循环结束后手动添加字符串结束符\0。此外，在约瑟夫问题的实现中，原程序因频繁压缩数组导致效率低下，经改进采用标记法后提升了运行效率。通过这次实验，我深刻体会到数组操作和指针处理的重要性，也认识到调试和逻辑分析在编程中的关键作用，今后将更加注重代码的健壮性和效率优化。